Interrupt-Guided Neural Code Completion with Static Analysis of Dependencies

Master Colloquium

Niklas Henrik Loeser | 3737339

2024-10-11

Heidelberg University
Institute of Computer Science
Artificial Intelligence for Programming (AIP)

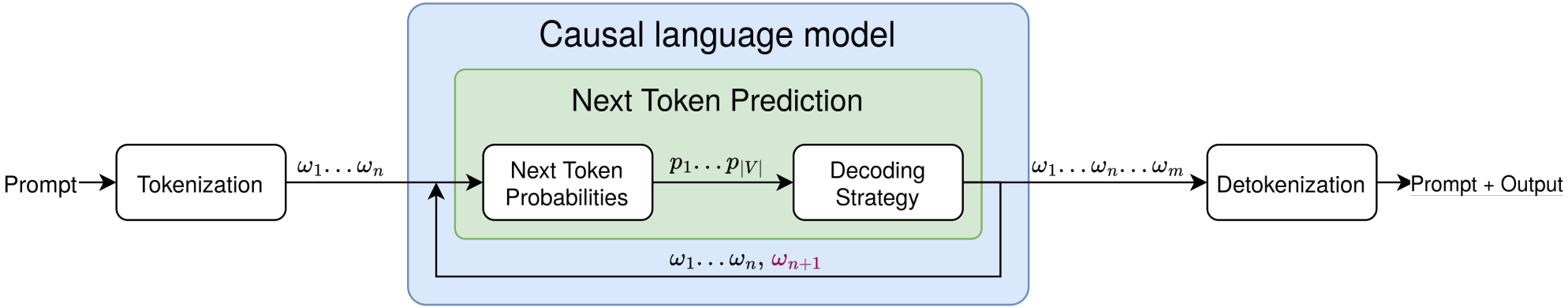
Prof. Dr. Artur Andrzejak

Contents

1. Introduction 3
2. Comparison to Related Work 11
3. Interrupt-Based Meta-Strategy 18
4. Full Approach 25
5. Results 28
6. Discussion 38
7. One-Click-Demo™ 46
8. Appendix 47

1. Introduction

1.1 Causal Language Models



- V: vocabulary of all tokens
- token: subword, word, or character
- $\omega_1 \dots \omega_n; \omega \in V$: sequence of tokens ($\Longrightarrow$ text)
- $p_1 \dots p_{|V|}; \sum_{i=1}^{|V|} p_i = 1$: probability distribution over V

1.2 Motivation

- Causal Language Models (CLMs) help reduce writing boilerplate code
- Copilot is widely used
- CLMs trained on data *until cut-off date*
- CLMs have issues with changing dependencies

1.3 Problem Scenarios

1. *Feature recently changed (deprecation)*
2. Feature recently added
3. Feature recently removed
4. Feature changed in any way, but not represented in training data

1.4 Interlude: Example Pydantic 1 - Copilot

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int


def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.

    Args:
        user (User): Pydantic user model

    Returns:
        Dict[str, Any]: User attributes as a Python key value mapping
    """
    # the function user.dict is deprecated, and user.model_dump should be used instead
    return user.dict()
```

Listing 1: Invalid completion by Copilot, even when guided by code comments.

1.5 Interlude: Example Pydantic 1 - Solution

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.

    Args:
        user (User): Pydantic user model

    Returns:
        Dict[str, Any]: User attributes as a Python key value mapping
    """
    return user.model_dump()
```

Listing 2: Gold solution highlighted in purple.

1.6 Goals

1. Retrieval: Retrieve accurate code context using IDE tooling
2. Injection: Inject context into model, to guide it …
   - … away from incorrect suggestions (deprecated / removed features)
   - … towards correct suggestions (new features)
3. Low performance overhead

1.7 Contributions

1. Static Analysis-guided neural code completion
   - Online monitoring of code completion
   - Accurate context retrieval
   - Hybrid injection of context using Monitor-Guided-Decoding (MGD)
2.
3.

1.7 Contributions

1. Static Analysis-guided neural code completion
   - Online monitoring of code completion
   - Accurate context retrieval
   - Hybrid injection of context using Monitor-Guided-Decoding (MGD)

2. Interrupt-Based Meta-Strategy (IBMS)
   - Inject context into the prompt “during” generation
   - $\Longrightarrow$ Low performance overhead
   - Applicable outside of code completion

3.

1.7 Contributions

1. Static Analysis-guided neural code completion
   - Online monitoring of code completion
   - Accurate context retrieval
   - Hybrid injection of context using Monitor-Guided-Decoding (MGD)

2. Interrupt-Based Meta-Strategy (IBMS)
   - Inject context into the prompt “during” generation
   - $\Longrightarrow$ Low performance overhead
   - Applicable outside of code completion

3. DependencyEval dataset
   - Handcrafted dataset to evaluate the approach used by the model
   - 25 Python tasks with fully pinned code environments
   - Evaluation framework to evaluate approach

2. Comparison to Related Work

2.1 Categories of Related Work

1. Prompt Injection Works
   - Single-step meta-strategies
   - Multi-step meta-strategies
2. Logit-based Approaches

2.2 Prompt Injection Works

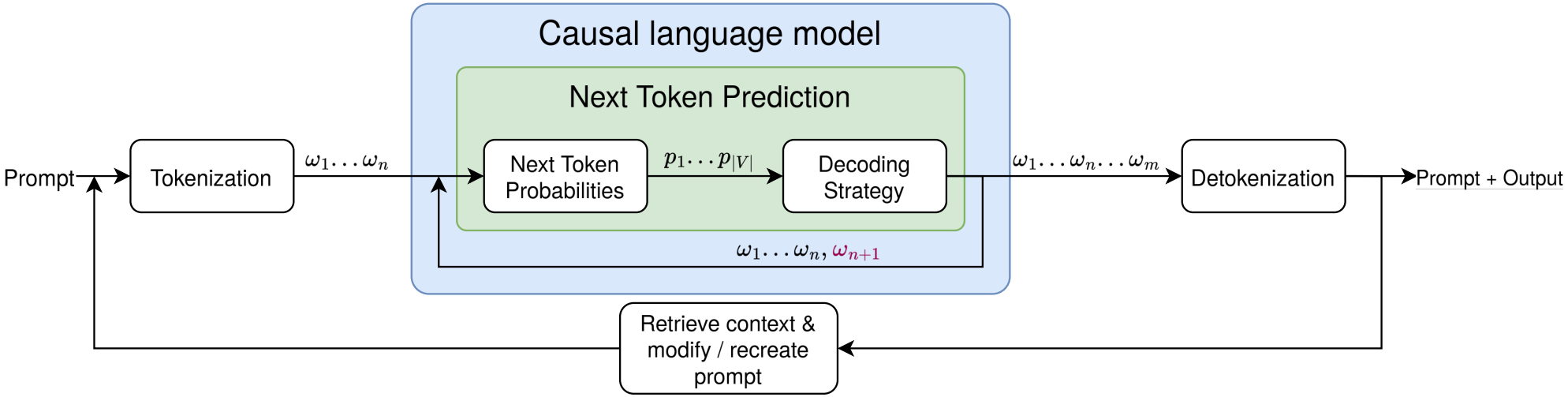


Figure 1: Additions to the causal language model to enable multi-step prompt injection.

2.3 Logit-based Approaches

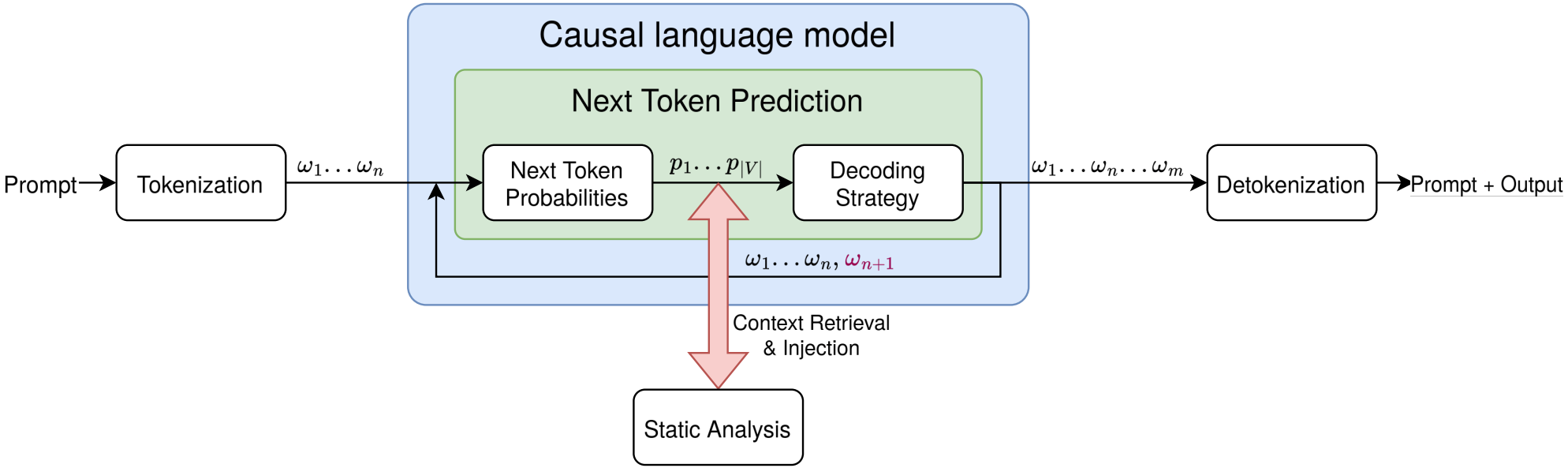


Figure 2: Additions to the causal language model to enable logit-based approaches.

2.4 DocPrompting [1]

- Key points
  - Single-step meta-strategy
  - Context source: Online Documentation (DevDocs)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
-
-

2.4 DocPrompting [1]

- Key points
  - Single-step meta-strategy
  - Context source: Online Documentation (DevDocs)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Flexible which documentation to embed
  - Documentation may include high quality user-submitted guides / examples
  - Fast
-

2.4 DocPrompting [1]

- Key points
  - Single-step meta-strategy
  - Context source: Online Documentation (DevDocs)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Flexible which documentation to embed
  - Documentation may include high quality user-submitted guides / examples
  - Fast
- Limitations
  - Context retrieved based on given code, *not to-be-generated code*
  - No differentiation between dependency sources

2.5 RepoCoder [2]

- Key points
  - Multi-step meta-strategy
  - Context source: Repository excluding environment files (venv, node_modules, etc.)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
-
-

2.5 RepoCoder [2]

- Key points
  - Multi-step meta-strategy
  - Context source: Repository excluding environment files (venv, node_modules, etc.)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Full source code as context, including docstrings
  - Bases retrieval on generated code
-

2.5 RepoCoder [2]

- Key points
  - Multi-step meta-strategy
  - Context source: Repository excluding environment files (venv, node_modules, etc.)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Full source code as context, including docstrings
  - Bases retrieval on generated code
- Limitations
  - Slower due to multiple steps
  - No retrieval from dependencies

2.6 Monitor-Guided-Decoding[1] [3]

- Key points
  - Online monitoring / manipulation of token sequence
  - Context source: IDE completion entries
  - Retrieval process: Request language server
  - Injection point: Next token probability distribution
-
-

2.6 Monitor-Guided-Decoding[1] [3]

- Key points
  - Online monitoring / manipulation of token sequence
  - Context source: IDE completion entries
  - Retrieval process: Request language server
  - Injection point: Next token probability distribution
- Benefits
  - Accurate context retrieval
  - Low performance overhead
-

2.6 Monitor-Guided-Decoding[1] [3]

- Key points
  - Online monitoring / manipulation of token sequence
  - Context source: IDE completion entries
  - Retrieval process: Request language server
  - Injection point: Next token probability distribution
- Benefits
  - Accurate context retrieval
  - Low performance overhead
- Limitations
  - Injection of unstructured context (docstring) hard
  - Only focuses on available code, should disallow deprecated code

3. Interrupt-Based Meta-Strategy

3.1 Goals

- Online injection of context into the prompt (e.g. docstrings)
- ⟹ Enable MGD to inject unstructured context
- Full resumption of generation after injection

  (do not regenerate current partial result[1])
- ⟹ Support beam search decoding strategy

[1] *Details massively simplified here*

3.2 Approach

- Add special “interruption” token to model / tokenizer
- Stop generation on interrupt token
- MGD sets interrupt token when context available
- Edit prompt with provided context (*location* / *scope*)
- Patched model generation to enable full resumption[1]

[1] *To fully enable beam search decoding*

3.3 Example

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.

    Args:
        user (User): Pydantic user model

    Returns:
        Dict[str, Any]: User attributes as a Python key value mapping
    """
    return user.
```

3.3 Example

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.

    Args:
        user (User): Pydantic user model

    Returns:
        Dict[str, Any]: User attributes as a Python key value mapping
    """
    # The following methods are deprecated, use alternatives as instructed in the given docstrings:
    # - The `dict` method is deprecated; use `model_dump` instead.
    # - The `json` method is deprecated; use `model_dump_json` instead.
    # - ...
    return user.
```

3.3 Example

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.

    Args:
        user (User): Pydantic user model

    Returns:
        Dict[str, Any]: User attributes as a Python key value mapping
    """
    return user.model_dump()
```

3.4 Refresher: Key points

- Inject context into the prompt “during” generation
- $\Longrightarrow$ Low performance overhead
- Applicable outside of code completion

4. Full Approach

4.1 Key points

- Hybrid approach using MGD and IBMS
- Retrieve context from IDE tooling in real-time:
  - Code completion entries
  - Signature help
- Downrank deprecated code completions
- Uprank other code completions
- IBMS is main tool to guide model, prediction manipulation secondary

4.2 Overview

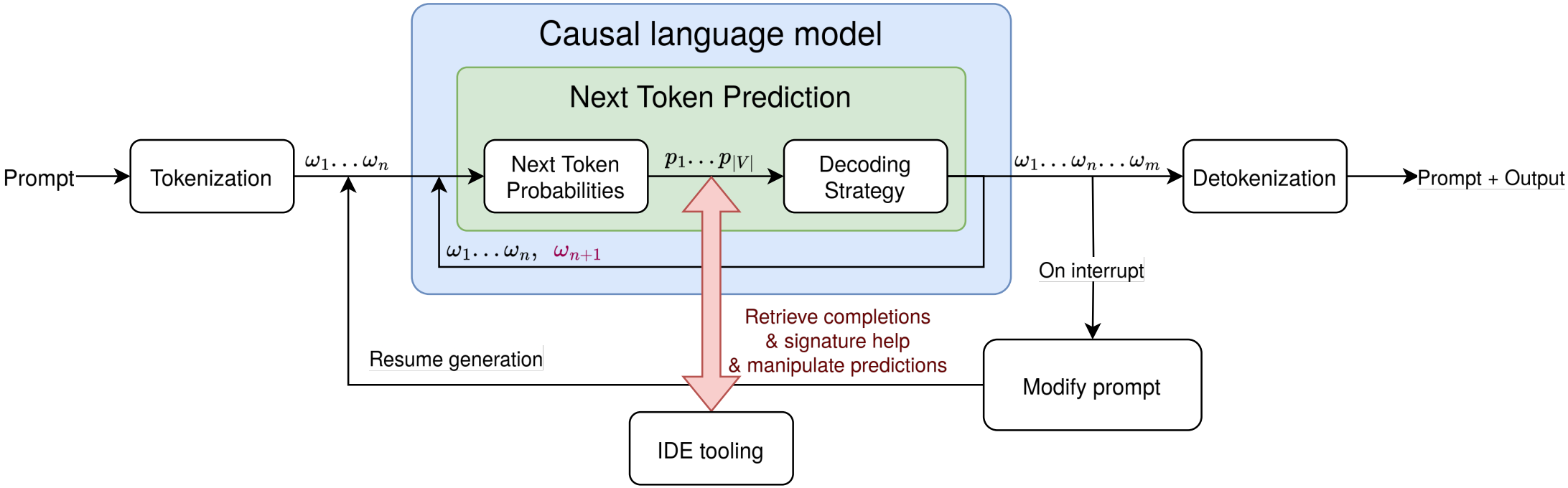


Figure 1: Additions to the causal language model to enable the approach.

5. Results

5.1 DependencyEval Dataset

- Handcrafted dataset to evaluate the approach used by the model
- 25 Python tasks with fully pinned code environments
- Evaluation framework to evaluate approach
- Captures tasks for different problem scenarios:
  - Feature recently changed (deprecation)
  - Feature recently added
  - Feature recently removed
  - Feature changed in any way, but not represented in training data

5.1 DependencyEval Dataset

JSON

```
{
  "task_name": "textual_1",
  "package_dependencies": [
    "textual==0.51.0"
  ],
  "reason": "The parameter tab_behavior was renamed",
  "kind": "modification",
  "date": "2024-02-15",
  "code_kind": "parameter",
  "modification_kind": "rename",
  "changelog": "https://github.com/Textualize/textual/blob/v0.51.0/CHANGELOG.md",
  "python_version": "3.8"
}
```

Listing 6: Example dataset task entry. Other fields omitted for brevity.

5.1 DependencyEval Dataset

Python

```python
class Test(TestCase):
    def test_output_correctness(self):
        app = App()
        out = create_textual_text_area_with_indent()
        assert isinstance(out, TextArea)
        assert hasattr(out, "tab_behavior")
        assert out.tab_behavior == "indent"

    def test_approach_correctness(self):
        app = App()
        TextArea = MagicMock(TextArea)
        globals()["TextArea"] = TextArea
        out = create_textual_text_area_with_indent()
        assert TextArea.call_count == 1
        kwargs = TextArea.call_args.kwargs
        assert "tab_behavior" in kwargs
        assert "tab_behaviour" not in kwargs
        assert kwargs["tab_behavior"] == "indent"
```

5.2 Experiment Environment

Component	Value
CPU	2x AMD EPYC 7713 64-Core
Memory	1 TB
GPU	4x NVIDIA GeForce RTX 3090
OS	Ubuntu 21.04
Kernel	5.15.0
Docker	20.10.7
Python	3.9.18
Transformers	4.39.0
PyTorch	2.2.1
CUDA	11.8

5.3 Experiment Setup

1. Models:
   - Code Llama Instruct 7B (CodeLlama)
   - Magicoder-S-DS 6.7B (Magicoder)
   - MistalHermes-CodePro 7B (MistalHermes)
   - Copilot (only for baseline)
   - *(each for 6 different decoding strategies)*
2. Baseline: Default model generation
3. Approach: Use one of the three configurations:
   - Default
   - Masked
   - Use completion

5.4 Baseline Results - Models

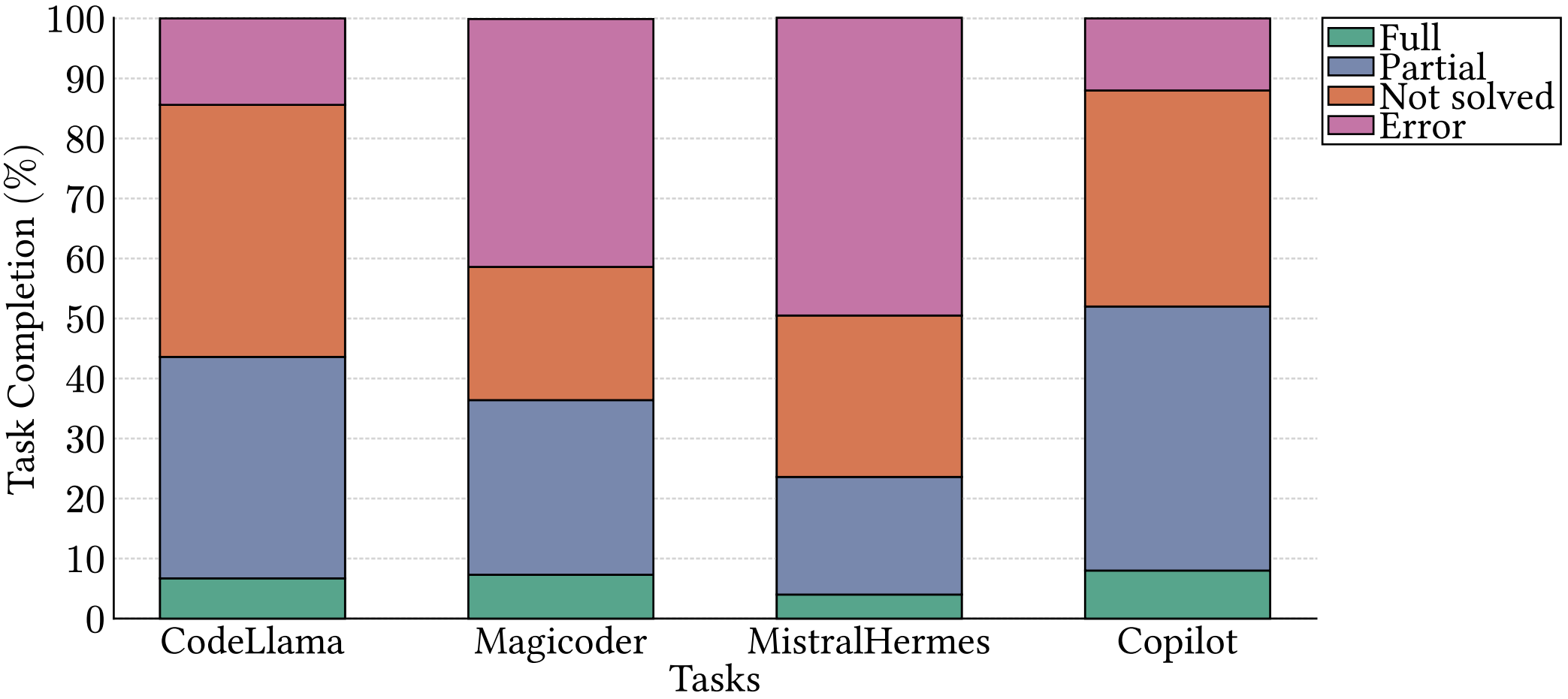


Figure 2: Baseline results for different models.

5.5 Baseline Results - Model configurations

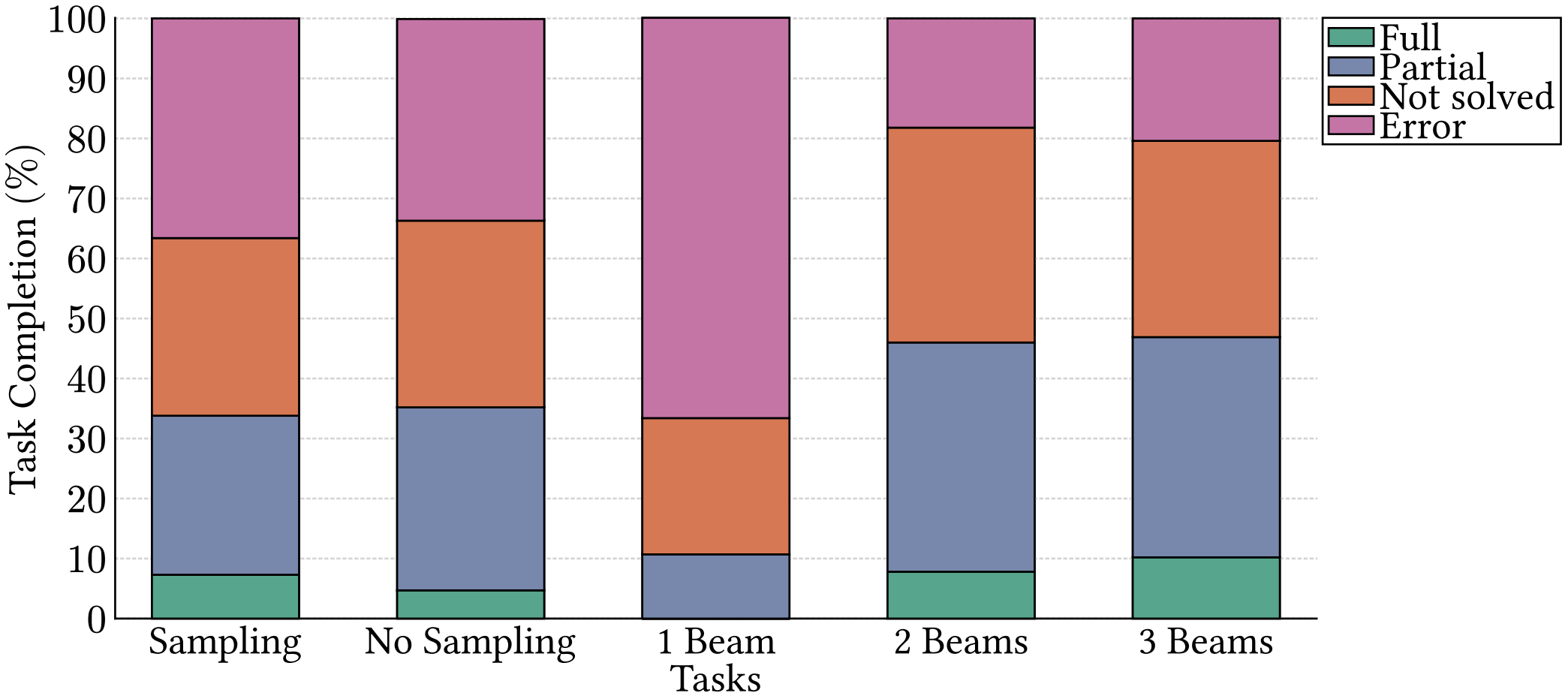


Figure 3: Baseline results for different model configurations.

5.6 Baseline Results - Task Scenarios

Name	Fully solved	Partially solved	Not solved	Error
pytorch_1	63.64%	12.73%	0%	23.64%
bidict_2	43.64%	20%	14.55%	21.82%
rich_2	20%	7.27%	40%	32.73%
pandas_1	14.55%	9.09%	16.36%	60%
dateutil_1	5.45%	32.73%	34.55%	27.27%
polars_1	3.64%	0%	65.45%	30.91%
pytorch_3	0%	72.73%	3.64%	23.64%
pytorch_2	0%	70.91%	0%	29.09%
dotted_1	0%	69.09%	5.45%	25.45%
pydantic_1	0%	67.27%	12.73%	20%
pydantic_3	0%	63.64%	10.91%	25.45%
sklearn_2	0%	56.36%	18.18%	25.45%
rich_1	0%	52.73%	25.45%	21.82%

Name	Fully solved	Partially solved	Not solved	Error
pydantic_2	0%	49.09%	23.64%	27.27%
numpy_1	0%	45.45%	7.27%	47.27%
textual_1	0%	30.91%	41.82%	27.27%
dotted_2	0%	25.45%	52.73%	21.82%
fastapi_1	0%	16.36%	41.82%	41.82%
bidict_1	0%	14.55%	52.73%	32.73%
emoji_1	0%	1.82%	74.55%	23.64%
textual_2	0%	1.82%	72.73%	25.45%
sqlalchemy_1	0%	0%	80%	20%
tsv2py_1	0%	0%	67.27%	32.73%
sklearn_1	0%	0%	0%	100%
theflow_1	0%	0%	0%	100%
Average	6.04%	28.8%	30.47%	34.69%

Figure 4: Baseline evaluation results for each task. The table is sorted from best to worst solved tasks. Copilot results are included.

5.6 Baseline Results - Task Scenarios

Name	Fully solved	Partially solved	Not solved	Error
pytorch_1	63.64%	12.73%	0%	23.64%
bidict_2	43.64%	20%	14.55%	21.82%
rich_2	20%	7.27%	40%	32.73%
pandas_1	14.55%	9.09%	16.36%	60%
dateutil_1	5.45%	32.73%	34.55%	27.27%
polars_1	3.64%	0%	65.45%	30.91%
pytorch_3	0%	72.73%	3.64%	23.64%
pytorch_2	0%	70.91%	0%	29.09%
dotted_1	0%	69.09%	5.45%	25.45%
pydantic_1	0%	67.27%	12.73%	20%
pydantic_3	0%	63.64%	10.91%	25.45%
sklearn_2	0%	56.36%	18.18%	25.45%
rich_1	0%	52.73%	25.45%	21.82%

Name	Fully solved	Partially solved	Not solved	Error
pydantic_2	0%	49.09%	23.64%	27.27%
numpy_1	0%	45.45%	7.27%	47.27%
textual_1	0%	30.91%	41.82%	27.27%
dotted_2	0%	25.45%	52.73%	21.82%
fastapi_1	0%	16.36%	41.82%	41.82%
bidict_1	0%	14.55%	52.73%	32.73%
emoji_1	0%	1.82%	74.55%	23.64%
textual_2	0%	1.82%	72.73%	25.45%
sqlalchemy_1	0%	0%	80%	20%
tsv2py_1	0%	0%	67.27%	32.73%
sklearn_1	0%	0%	0%	100%
theflow_1	0%	0%	0%	100%
Average	6.04%	28.8%	30.47%	34.69%

Figure 4: Baseline evaluation results for each task. The table is sorted from best to worst solved tasks. Copilot results are included.

5.7 Approach Results - Task Scenarios

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
pydantic_1	+44.44%	−51.85%	+1.85%	+5.56%	48.15%
textual_1	+25.93%	−31.48%	−24.07%	+29.63%	44.44%
pydantic_3	+25.93%	−44.44%	+12.96%	+5.56%	55.56%
rich_2	+24.07%	−3.7%	−37.04%	+16.67%	59.26%
dateutil_1	+14.81%	−1.85%	−27.78%	+14.81%	70.37%
pydantic_2	+9.26%	−31.48%	+7.41%	+14.81%	68.52%
pandas_1	+3.7%	+5.56%	±0%	−9.26%	90.74%
rich_1	+1.85%	−14.81%	−3.7%	+16.67%	81.48%
textual_2	±0%	+1.85%	−20.37%	+18.52%	79.63%
emoji_1	±0%	+1.85%	−24.07%	+22.22%	75.93%
sklearn_1	±0%	±0%	±0%	±0%	100%
theflow_1	±0%	±0%	±0%	±0%	100%
bidict_1	±0%	±0%	−7.41%	+7.41%	92.59%
sqlalchemy_1	±0%	±0%	−9.26%	+9.26%	90.74%
tsv2py_1	±0%	±0%	−18.52%	+18.52%	81.48%
bidict_2	±0%	−7.41%	+1.85%	+5.56%	92.59%
dotted_2	±0%	−9.26%	−14.81%	+24.07%	75.93%
fastapi_1	±0%	−16.67%	−9.26%	+25.93%	74.07%
dotted_1	±0%	−20.37%	+1.85%	+18.52%	79.63%
numpy_1	±0%	−25.93%	+18.52%	+7.41%	74.07%
pytorch_2	±0%	−27.78%	+1.85%	+25.93%	72.22%
pytorch_3	±0%	−31.48%	+9.26%	+22.22%	68.52%
sklearn_2	±0%	−55.56%	+46.3%	+9.26%	44.44%
polars_1	−3.7%	±0%	−31.48%	+35.19%	64.81%
pytorch_1	−37.04%	+27.78%	±0%	+9.26%	62.96%
Average	+4.37%	−13.48%	−5.04%	+14.15%	73.93%

Figure 5: Baseline evaluation results for each task. The table is sorted from best to worst solved tasks. Copilot results are included.

5.7 Approach Results - Task Scenarios

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
pydantic_1	+44.44%	−51.85%	+1.85%	+5.56%	48.15%
textual_1	+25.93%	−31.48%	−24.07%	+29.63%	44.44%
pydantic_3	+25.93%	−44.44%	+12.96%	+5.56%	55.56%
rich_2	+24.07%	−3.7%	−37.04%	+16.67%	59.26%
dateutil_1	+14.81%	−1.85%	−27.78%	+14.81%	70.37%
pydantic_2	+9.26%	−31.48%	+7.41%	+14.81%	68.52%
pandas_1	+3.7%	+5.56%	±0%	−9.26%	90.74%
rich_1	+1.85%	−14.81%	−3.7%	+16.67%	81.48%
textual_2	±0%	+1.85%	−20.37%	+18.52%	79.63%
emoji_1	±0%	+1.85%	−24.07%	+22.22%	75.93%
sklearn_1	±0%	±0%	±0%	±0%	100%
theflow_1	±0%	±0%	±0%	±0%	100%
bidict_1	±0%	±0%	−7.41%	+7.41%	92.59%

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
sqlalchemy_1	±0%	±0%	−9.26%	+9.26%	90.74%
tsv2py_1	±0%	±0%	−18.52%	+18.52%	81.48%
bidict_2	±0%	−7.41%	+1.85%	+5.56%	92.59%
dotted_2	±0%	−9.26%	−14.81%	+24.07%	75.93%
fastapi_1	±0%	−16.67%	−9.26%	+25.93%	74.07%
dotted_1	±0%	−20.37%	+1.85%	+18.52%	79.63%
numpy_1	±0%	−25.93%	+18.52%	+7.41%	74.07%
pytorch_2	±0%	−27.78%	+1.85%	+25.93%	72.22%
pytorch_3	±0%	−31.48%	+9.26%	+22.22%	68.52%
sklearn_2	±0%	−55.56%	+46.3%	+9.26%	44.44%
polars_1	−3.7%	±0%	−31.48%	+35.19%	64.81%
pytorch_1	−37.04%	+27.78%	±0%	+9.26%	62.96%
Average	+4.37%	−13.48%	−5.04%	+14.15%	73.93%

Figure 2: Baseline evaluation results for each task. The table is sorted from best to worst solved tasks. Copilot results are included.

6. Discussion

- Which assumptions does the approach hold about the model?
- Could the evaluation have included tasks of the standard library?
- What is the difference between multi-step meta-strategies and the IBMS?
- Which further enhancements would improve the approach?
- Which assumptions does the approach hold about the programming language?

6.1 Implementation difficulties

1. Model generating unwanted tokens (repetitions; descriptive sentences after generating code)
2. Runtime does not provide enough control over decoding internals
3. *Beam search*:
   - sequence reordering
   - separate state for different sequences
4. Detokenization not inverse of tokenization in practice
5. Model ignoring injected context

6.1 Implementation difficulties

6. Use of different virtual environments

7. Language servers not providing accurate deprecation information

6.2 Baseline Results - Best combinations

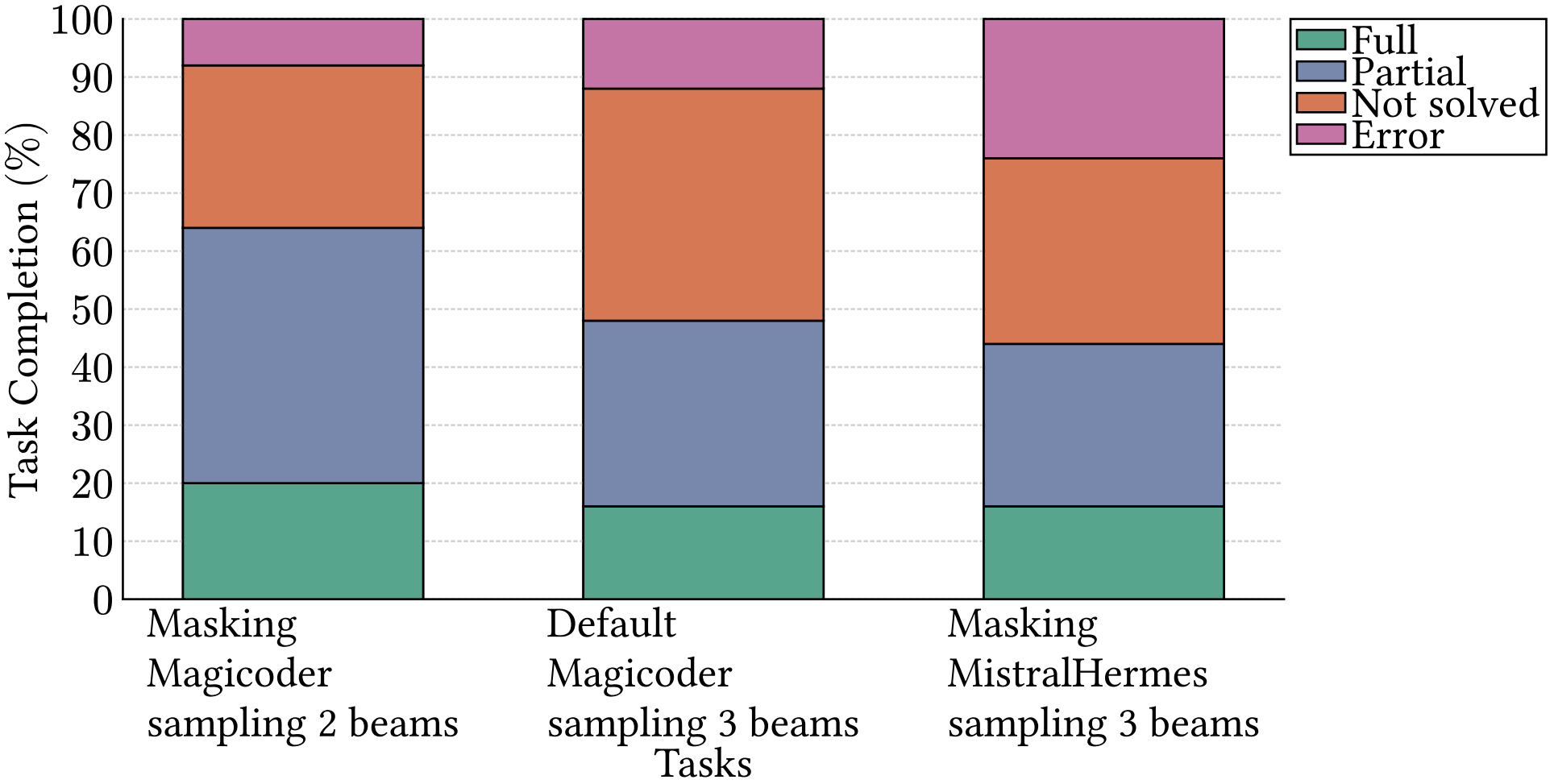


Figure 3: Overview over the top 3 baseline configurations.

6.3 Approach Results - Best combinations

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
Default CodeLlama sampling 2 beams	+20%	−16%	−12%	+8%	72%
Masking MistralHermes sampling 2 beams	+16%	−8%	±0%	−8%	84%
Masking CodeLlama sampling 2 beams	+16%	−12%	−4%	±0%	84%

Table 1: Overview over the top 3 approach configurations.

6.4 Baseline Results - Worst combinations

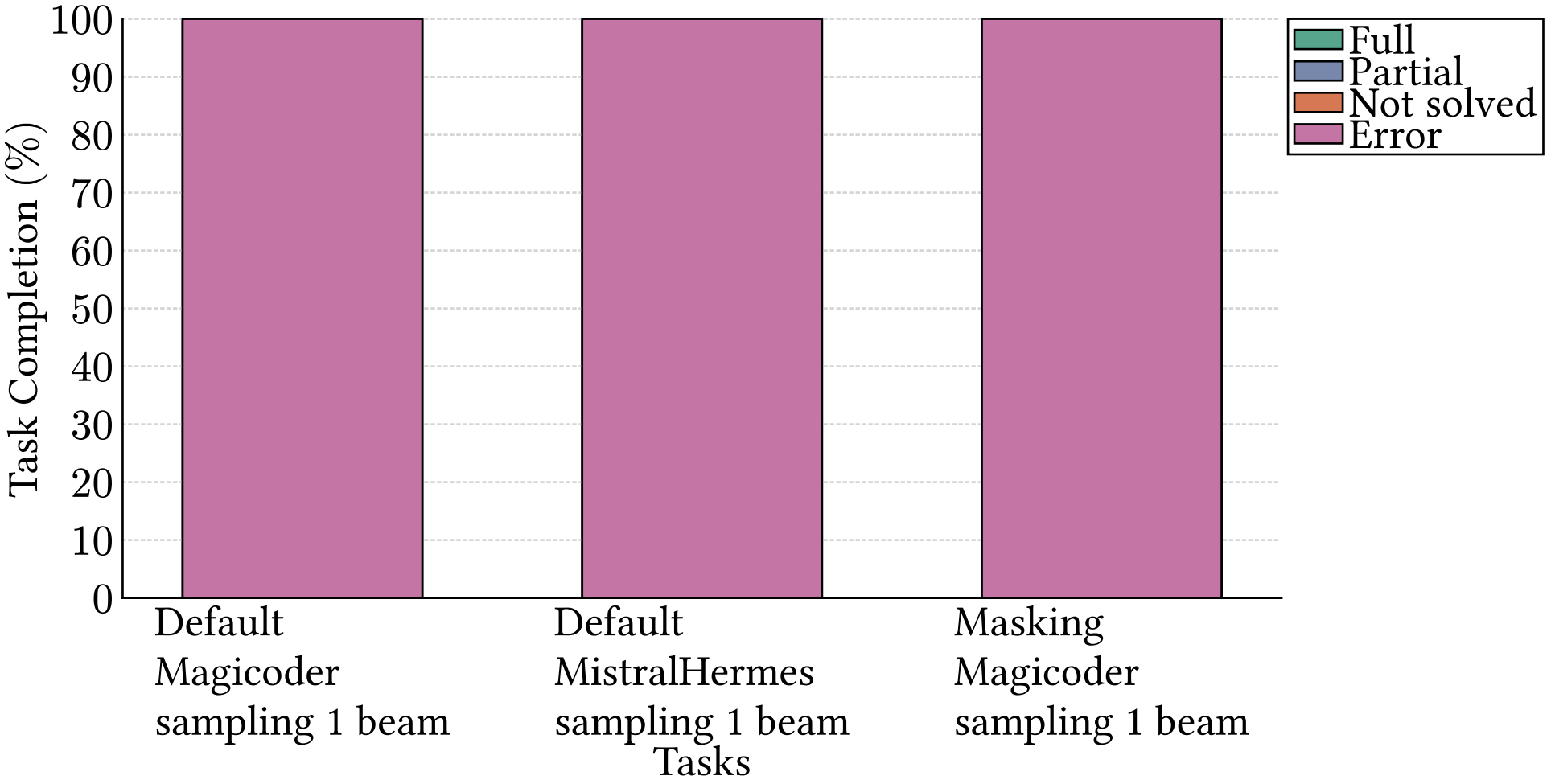


Figure 4: Overview over the top 3 baseline configurations.

6.5 Approach Results - Worst combinations

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
Use Completions Magicoder greedy 3 beams	−8%	−48%	±0%	+56%	44%
Use Completions CodeLlama greedy 2 beams	−4%	−36%	±0%	+40%	60%
Use Completions MistralHermes greedy 2 beams	−4%	−20%	−32%	+56%	44%

Table 2: Overview over the top 3 approach configurations.

7. One-Click-Demo™

8. Appendix

8.1 Acronyms

CLM: Causal Language Model

IBMS: Interrupt-Based Meta-Strategy

MGD: Monitor-Guided-Decoding

8.2 Bibliography

[1] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, “DocPrompting: Generating Code by Retrieving the Docs.” Accessed: Jun. 23, 2024. [Online]. Available: http://arxiv.org/abs/2207.05987

[2] F. Zhang *et al.*, “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation.” Accessed: Dec. 05, 2023. [Online]. Available: http://arxiv.org/abs/2303.12570

[3] L. A. Agrawal, A. Kanade, N. Goyal, S. K. Lahiri, and S. K. Rajamani, “Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context.”